

Automate FW testing with MDBCore scripting, part of the MPLABX ecosystem (introduction)

Presenter Stefan Luethin

Nov 2023

Agenda

1: what is the *MPLABX* ecosystem *MDBCore*, *SDK* -> how to use for automated FW testing

2: compile without IDE

3: Automated *FW-testing* with different *MDBCore* use-models

- a. simple *mdb* -> DOS/bash-script, easy to use but has limits
- b. more advanced *mdbcs* -> *.jar module needs scripting-language with Java-interface, more complex but solves *mdb* limits and opens door into Java-universe
- c. full, but most complex usage of *MDBCore* with Java -> SDK

4: summary

HW- and SW-Tools for demo

- **HW**

- ATSAM21-xpl (-> [atsamd21-xpro](#))
- EDBG (onboard debugger used – no extra tool required)

- **SW**

- MPLABX® v6.15 (get from [Microchip/MPLABXdownloads](#))
- XC32-v4.35 (from: [XC32compilers](#))
- MPLABX® SDK v6.15 (from: [mplab-x-sdk-mplab-x-ide](#) -> SDK-version must match MPLABX-IDE version!!)
- NetBeans with JDK v8.2 (from [NB+JDK-v8.2](#) -> *latest version not tested but should work*)
- Groovy-v2.5.8 (from [groovy.org](#))

Motivation

- **Motivation - why to do FW-test-automation?**

- How to ensure **FW-quality** over **multiple HW-platforms** and **FW-revisions** ?
- How to track **progress** of FW-project to keep milestones? How much **automation** today and @what **cost** ?

- **What you will learn**

- All actions in IDE (=interactive) are available in a scripting environment through the MDBCore-framework
->compile **and** debug from script in an automated way instead of manually in the IDE
- Where, how will the automation help?
 - **Regression** to automatically run a set of tests, eg on different HW-platforms **OR** different SW-releases
 - **Calibration**: system with 5x registers each $1 \leq \text{regVal} \leq 10$ -> optimal seed given by building the average of mem[0-1024] and the closer it is to 0.5 the better...-> how to find best 'set-of-values' to calibrate your system?
 - Or **Fault-Injection** or ...
- With this automation you can
 - automatically ensure **FW-quality** with continues checking
 - automatically check a certain FW-revision works on **all HW-platforms**
 - track progress of your FW-development to keep/ensure **project-milestones**
 - **All this @no cost, as it is included in the Microchip Ecosystem MPLAB®X**
- **Target audience**: FW-tester, Quality or Project-Leads
- **Knowledge**: working experience of MPLAB®X & any OO-language knowledge helpful

FW testing with MPLABX

What is FW-testing?

The screenshot displays the MPLAB X IDE v6.00 interface for a project named 'CICDgit_same70xplp_sim'. The main editor shows a C source file 'src/main.c' with the following code:

```
46 volatile uint32_t char2prntIdx; //- index for 'charArray'
47 void testEnd(void);
48
49
50 void sysTickTimeout_handler(uintptr_t context)
51 {
52     sysTickTmrExFlag=true;
53 }
```

The 'Projects' pane on the left shows the project structure, including 'Header Files', 'Important Files', 'Linker Files', 'Source Files', 'config', 'main.c', 'Libraries', and 'Loadables'. The 'CDgit_same70xplp_sim' project is selected, showing its configuration: 'Project Type: Application', 'Device: ATSAME70Q21B', 'Checksum: Debug Image', 'CRC32: Hex file unavailable', 'Packs: SAME70_DFP (4.6.102), CMSIS (5.8.0)', 'Compiler Toolchain: XC32 (v4.10)', 'Debug Image: ELF', 'Device support information: SAME70_DFP (4.6.102)', 'License: Workstation - cpp - HPA: 7087 day(s)', 'Linker Reserved Heap: 512 (0x200) bytes', 'Linker Reserved Program: 2097152 - Used 10', 'Memory: Data 393,216 (0x60000) bytes, Data Used: 1,983 (0x7BF) Free: 391,233, Program 4,194,304 (0x400000) bytes, Program Used: 10,539 (0x292B) Free: 4,183,765 (0x3F6873)', 'Debug Tool: Simulator', 'Click for Simulated Peripherals', 'Debug Resources: Program BP Used: 1 Free: 999, Data BP Used: 1 Free: 999'.

The 'Output' pane at the bottom shows a list of variables: 'tmrExFlag', 'char2prntIdx', 'ledCnt', 'sysTickTmrExFlag', 'myCnt', 'PIOA_REGS', 'PIOA', 'sysTickTmrExFlag', 'myCnt', and '<Enter new watch>'. The 'Data Memory' pane on the right shows a table of memory addresses and their values:

Address	00	04
2040_0000	00000000	00000000
2040_0010	00000000	00000000
2040_0020	00000000	00000000
2040_0030	00000000	00000000
2040_0040	00000000	00000000
2040_0050	00000000	00000000
2040_0060	00000000	00000000
2040_0070	00000000	00000000
2040_0080	00000000	00000000
2040_0090	00000000	00000000
2040_00A0	00000000	00000000
2040_00B0	00000000	00000000

Motivation to add automation
and this class will teach you
how to use **MDBCore** for this

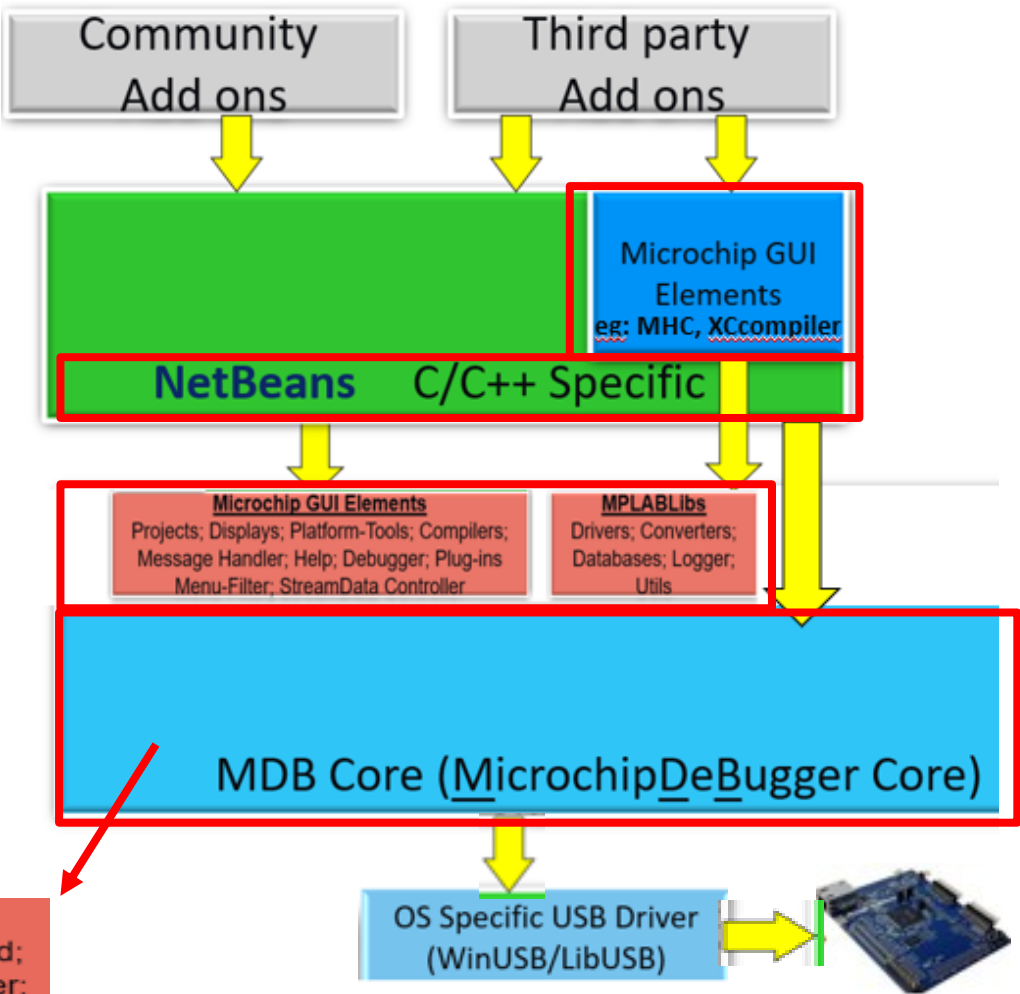
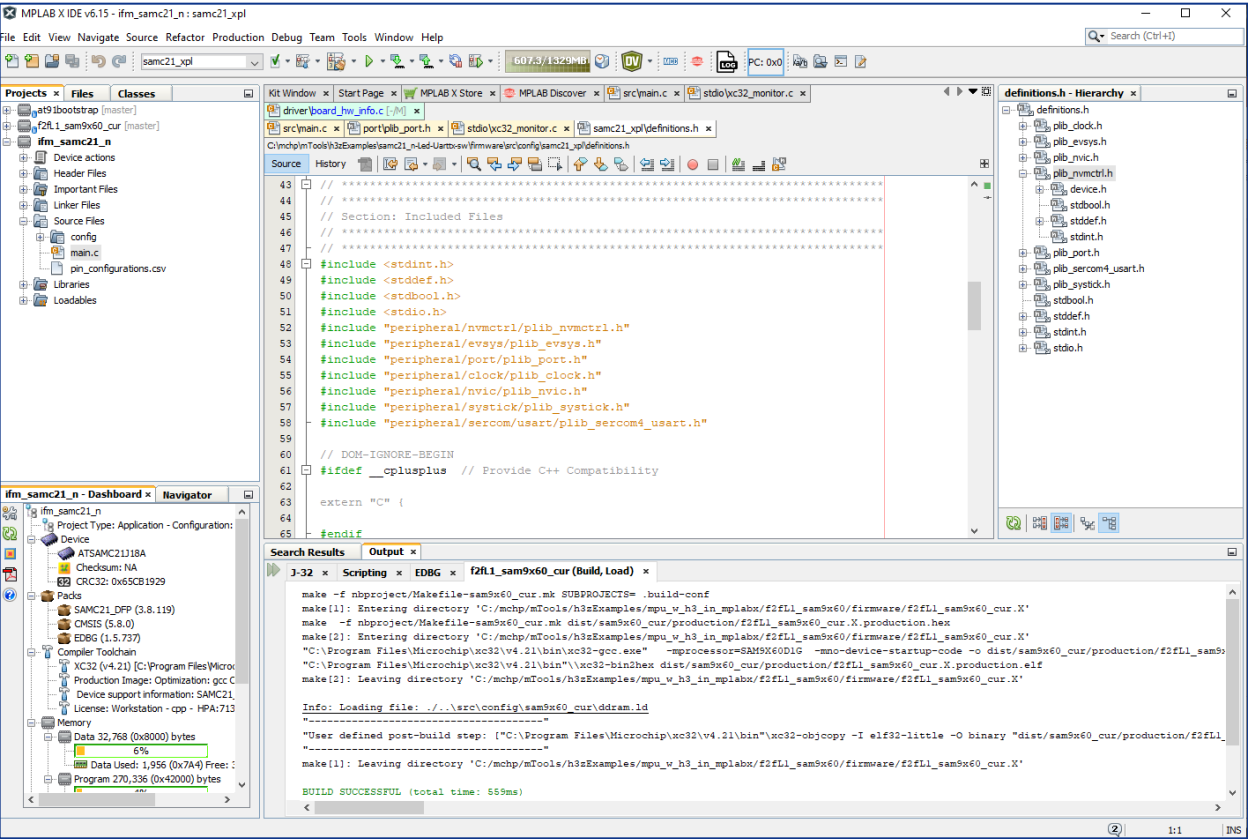
- >running FW & decide works/or not
- >breakpoints
- >variables values
 - ok: $0 \leq \text{VarX} \leq 5$ -> fail: if $\text{VarX} > 5$
- >read memory
- >write variables (double-click)
- ...
- >ok for 2x tests, but with x00?
- >test10 start depends on test9
- >calibration: best seed for $0 < x[5] < 9$??

Glossary, Terminology

Quick summary of terms used and next slides shows where each one fits in

- *mdb* = Microchip DeBugger
- *MDBCore*
 - collection of many (>1000) JAVA classes -> **one pillar** of MPLAB[®] X
 - Need Java to use MDBCore = complexity
- *mdb.bat*
 - DOS-script with java-call using *mdb.jar* (=MDBCore module)
 - Commandline interface to **program** and **debug** devices -> easy-to-use but with limits
- *mbcs*
 - MDBCore Scripting=mbcs -> *mbcs.jar*
 - requires scripting language with Java-interface and some setup
 - *mbcs* **simplifies** MDBCore usage **and** solves *mdb.bat* limits **and** opens door to Java-universe
- *Plug-in*
 - A visual extension to MPLAB[®] X like DataVisualizer, Harmony,...

Architecture breakdown – what/where is MDBCore?



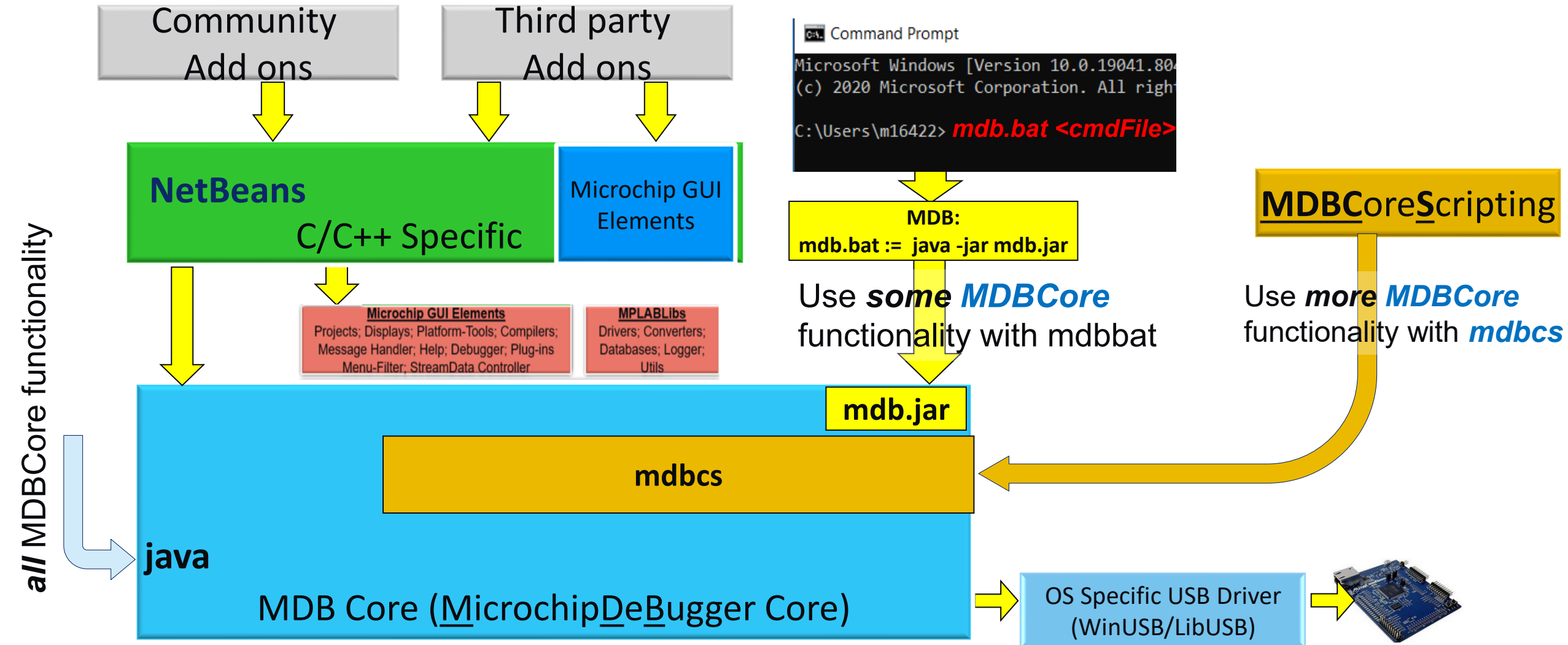
MDB CORE (MicrochipDebugger)
Assembly; Checksum; CodeGuard; Debugger; Program; ControlPointMed; FreezeMed; MessageMed; ToolExecMed; Memory; Loader; ObjFileParser; ICD2; ICD3; PICKit3; PK3OB; PICKit2; REALICE; PM3; SKDE; Simulator; TargetmemTransfer; OPTSupport; StreamingDataProcessing; SymbolViewRes; RegisterViewRes; Translator; UsermemUtil; Disassembly

MDBCore use-cases overview

Summary



- = ***some*** MDBCore-functionality
- = ***more*** MDBCore-functionality
- = ***full*** MDBCore-functionality



MDBCore use-cases introduction

MDBCore use cases – a short introduction

➤ **mdb** (->part3.1):

- Easiest **MDBCore**-usecase
- location `$mplab\bin\mdb.bat`
- `mdb.bat` calls (`mdb.jar` = MDBCore-module):
`$jdkhome\java -jar $mplab\lib\mdb.jar`
- Usage
 - **interactively:**
`(DOS)> mdb.bat`
`> help`
 - with **commandsfile:**
`(DOS)> mdb.bat mytest1.txt`
- `mdb.bat` has limits
- but in every MPLAB-installation = no installation effort

=> will look at **mdb** in part3.a

```
C:\Windows\System32\cmd.exe - mdb.bat

C:\mchp\Masters\2021_eMasters-MU\24024DEV11_eMasters\eMaster_MU-version3>mdb.bat
>
>help

The list of classes of commands:

breakpoints -- Making program stop at certain points
data -- Examining/Changing data
deviceandtool -- Selecting debug tool and device
networktool -- Network Tools
others -- Others
programming -- Programming device and its relative functions
running -- Running the program
stack -- Examining stack

Type "help" followed by a class name for a list of commands in that class.
Type "help" followed by command name for full documentation.
Type "help" followed by "doc" to see how to open MDB documentation.
```

```
*C:\mchp\Masters\2021_eMasters-MU\24024DEV11_eMasters\labs\lab1_mdbbat\2_MDBsame70--mdbbat--ex--cmdFile.txt

File Edit Search View Encoding Language Settings Tools Macro Run Plugins Window ?

2_MDBsame70-mdbbat-ex-cmdFile.txt
1  #-- cmd-file for mdb.bat
2  device.ATSAME70Q21B
3  set.communication.interface.swd
4  hwtool.EDBG
5  reset
6
7  pwd
8  program."aPath\fwDev11_same70_xult.X.debug.elf"
9
10 break.endOfTest
11 info.break
12 run
13
14 halt
15 wait
16 print.testResult
17
18 #-and.quit.session
19 quit
```

MDBCore use cases

MDBCore use cases (continued)

➤ **mdbcs** (->part3.2):

- **mdbcs.jar** -> scripting w Java-interface (eg: Groovy)
- **mdbcs.jar** build from sources
 - > Netbeans-prj in SDK (below)
 - > complexity **mdbcs** >> **mdb.bat**
 - > **more** features of **MDBCore**
- **SDK** -> info & Netbeans-prj to build **mdbcs**
- **mdbcs.jar** scripting with Java-IF
- It solves **mdb.bat** limits of (**if-else**, **stdOut**)
- opens path into Java-universe (eg: **swingUIs**)
- 3rd party company **LDRA** (dynamic-software testing) use **mdbcs**

=> will look at **mdbcs** in part3.b

mdbcs.gr

```
// -Groovy script (procedural style)
package myTest1Pck
import com.microchip.mdbcs.Debugger
println "   ###: Creating debugger"
debugger = new Debugger("SAML21","EDBG", true)
res=debugger.connect() //='0' if successful
if(!res)
    debugger.disconnect()
    debugger = null
else
    debugger.loadFile("myprog.elf")
    debugger.program()
```

mdb.bat

```
1  #-- cmd-file for mdb.bat
2  device SAML21
3  set communication.interface swd
4  hwttool EDBG
5  #-- load+program *.elf
6  program "myprog.elf"
7
8  break endOfTest
```

MDBCore use cases

MDBCore use cases (continued)

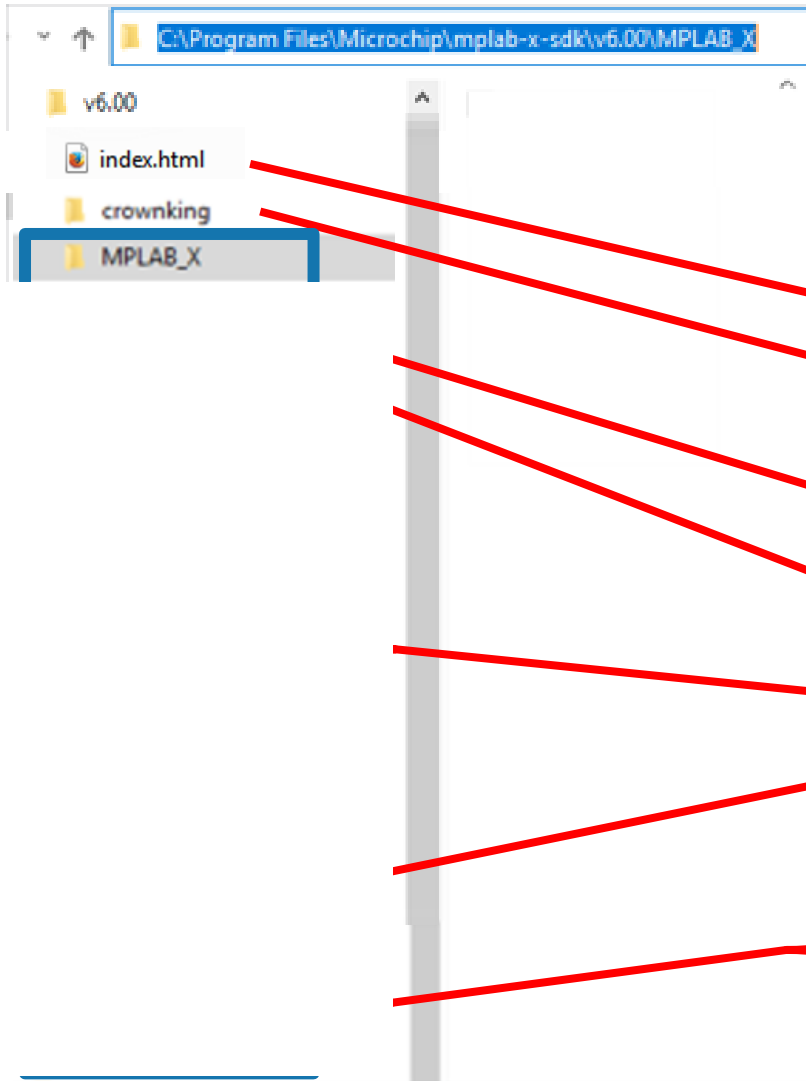
➤ MDBCore /Java (->part3.3):

- Access complete *MDBCore* framework (>1000 classes) -> need Java = complexity !
- 3rd party company *Razorcat*^(l) use *MDBCore* Java-adapter into their dynamic-software testing *Tessy*[®]

=> will look at *Java/MDBCore* and the *SDK* in part3.c

What's in the SDK

The MPLAB SDK



Get it from www.opensource4pic.org

and install zip (eg: `mplab-x-sdk-v6.15.zip`) into any path – recommended path:
`C:\Program Files\Microchip\mplab-x-sdk\<version>`

`Index.html` -> documentation (next slide)

`crownking\` -> deviceDB (= DFP)

`MPLABX\` -> the SDK

`MPLABX\doc` -> docu in html-format

`MPLABX\mdb` -> Netbeans-prj to build your own ***mdb.bat***

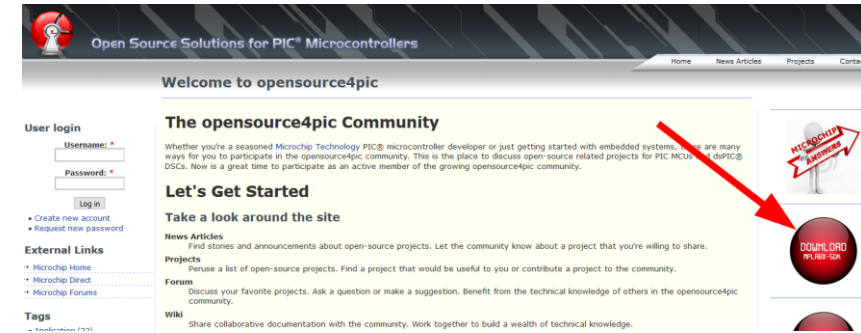
`MPLABX\mdbcs` -> Netbeans-prj for ***mdbcs***

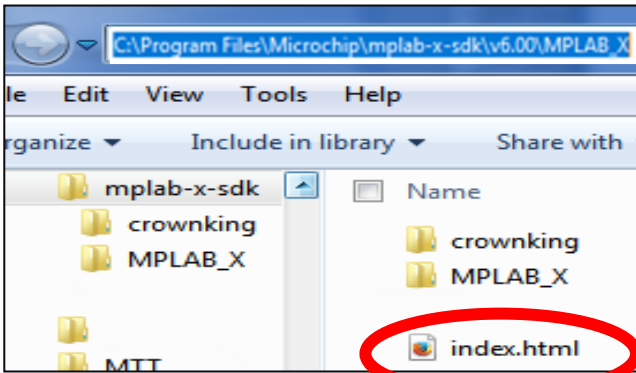
`MPLABX\utils` -> utility scripts, eg: to configure your OS (->later)

`MPLABX\mplab-core` -> Netbeans-prjs for MDBCore functionality eg: `fileParsing`

`MPLABX\mplab-ide` -> Netbeans-prjs for MPLAB-IDE functionality eg: `xc32-toolchain`

`MPLABX\sample` -> Netbeans-prjs with MDBCore applications – eg: using debug-API





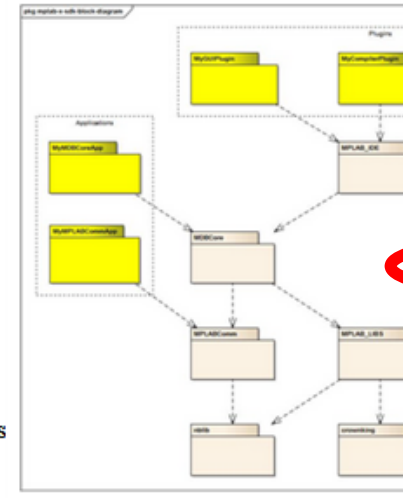
For MPLAB X IDE v5.45

This software development kit contains *documentation* and *sample code* for developing:

- A **General-Purpose (GUI) plugin** for MPLAB X IDE
- A **compiler plugin** for MPLAB X IDE (integrates a compiler)
- A **tool plugin** for MPLAB X IDE (integrates a hardware debugger, programmer, or simulator)
- An **application of MDBCore** (e.g., a bench test, a command-line debugger)

The diagram on your right illustrates where each kind of software plugin/application *interfaces* with the MPLAB X Platform, as well as the *dependencies* among the largest organizational units in the MPLAB X Platform.

NOTE: Device files are no longer installed with the SDK. Currently they can be found in MPLABX under the packs directory. For example: MPLABX/v5.45/packs.



- [Installing and Configuring the Development Tools](#)
- [Creating a New Plugin Project](#)
- [Plugin User Experience Guidelines](#)
- [NetBeans Docs & Support](#) - (External link)
- [General-Purpose Plugin Tutorial](#)
- [Compiler Plugin Tutorial](#)
- [Tool Plugin Tutorial](#)
- [Signing MPLABX 3rd party plugins](#)
- [Extend Wizard Tutorial](#)
- [Simulator Programmer's Guide](#)
- [MDBCore API User's Guide](#)
- [MDBCS: Example on using MDBCore APIs](#)
- [MPLAB Communications Library Documentation](#)

- [How to Build the Samples](#)
- [MDBCore Sample "query tools"](#) - Gets the names of all connected tools
- [MDBCore Sample "connect tool"](#) - Connects to the REAL ICE
- [MDBCore Sample "program device"](#) - Programs, runs, and halts a device
- [Debug-Aware Plugin Sample](#) - A window with project/debugger awareness
- [Extend Wizard Sample](#) - Extend the new project wizard

- [Source for "mdb"](#) - The Microchip command-line debugger built upon MDBCore
- [Source for the C30 Compiler Plugin](#) - The Microchip C30 compiler plugin

- [MPLAB IDE Java API Reference](#)
- [MDBCore Java API Reference](#)
- [crowningx Java API Reference](#)
- [MDBCS Java API Reference](#)
- [J2ME AB Communication Library Java API Reference](#)
- [crowning Java API Reference](#)
- [NetBeans v8.0.1 Java API Reference - \(External link\)](#)

Interesting for *mdbcs* section

Knowledge Check MDBCore+SDK intro

Summary and knowledge check module MDBCore-introduction

- How does it all fit together, Netbeans and MPLAB^(R)X and MDBCore?
=> MPLABX == Netbeans + plugins (eg: UI/MCC, headless/XC) + many Java*-libs and one is MDBCore (eg: debugger,...)
- What is the MPLAB^(R)X SDK?
=>a development platform with examples how to use MDBCore and NB-projects for mdbbat and mdbcs
- What MDBCore usecases did you hear about?
=>mdb.bat (3.1) = DOS/BAT-scripting to access **some** MDBCore-features (**some** limits but **easy**)
=>mdbcs (3.2) = scripting with Java-IF to get **more** MDBCore-features (**no** limits but **more** complex)
=>java (3.3) = use **all** MDBCore-features with Java directly (**complex**)
- What are the mdb.bat limits and how to solve them?
=>eg 'no looping' and 'no if-else' in the mdb.bat-commandfile
=>can only print result to stdout but not interpret it
=>one MCHP-supported way to solve mdbbat-limits is mdbcs, though you will
other solutions on the web that wrap mdbbat into python (at your own risk)

End Part1

End of Part1:

what is the *MPLABX* ecosystem *MDBCore*, *SDK* -> how to use for automated FW testing

Next/Part2:

compile outside the IDE